

HERMES / ARCHITECTURE BRIEF

Memory System

v2-only · BGE-large · shared skills · per-agent visibility

Decisions, evidence, patches, and follow-ups
from the one-day overhaul of the Hermes memory layer.

2026-05-17 · OpenClaw · prepared for technical review

Five wins. One root cause. Durable.

01

Committed to v2 plugin

v1 MemOS server retired — no Qdrant / Neo4j to babysit

02

BGE-large embeddings

1024-dim, local, +14% MRR over MiniLM at no API cost

03

Auto-promoter killed

orphan Sprint-3 cron writing share_scope=local every 15 min — disabled

04

Per-agent viewer toggle

AsyncLocalStorage + X-As-Profile, no second daemon needed

05

Open-world speaker attribution

names come from [handles] and self-intros, not a closed roster

FINAL STATE

36 / 36 stress-test checks green · 13 / 13 plugin patches verified · durable via postinstall-patches.sh

All work re-applies automatically after npm install/update of @memtensor/memos-local-plugin.

T H R E E S Y S T E M S · O N E W A S D O I N G T H E W O R K

The diagnosis

Part 1

Three things were being conflated

We had three memory ideas in play with similar names. The team wasn't sure which one was doing the work.

SYSTEM 1 MemOS v1 server IDLE Standalone HTTP service on :8001. Qdrant + Neo4j + SQLite. Cube-based isolation enforced server-side via API-key → user_id binding. The original production target.	SYSTEM 2 v2 memos plugin ACTIVE Hermes-native plugin. Single SQLite + UI on :18800. Reward backprop, L1/L2/L3 layers, automatic skill crystallization. Isolation by row-level namespace tuple.	SYSTEM 3 The hub PLANNED Future cross-machine sync feature. Config schema and a share_scope='hub' value exist; zero code reads them. Failing systemd unit was trying to start it.
---	--	---

What's actually running

Component	Role	Status	Location
v2 memos-local-plugin	Sole active memory provider	ACTIVE	~/.hermes/memos-plugin/ . :18800
MemOS v1 server	HTTP API; ceo-cube empty (zero writes)	STOPPED	localhost:8001 (disabled today)
memos-toolset	v1 client; plugin.yaml.disabled-2026-05-12	INERT	~/.hermes/plugins/memos-toolset/
memos-hub.service	Tried to start non-existent hub mode	DISABLED	scripts/migration/bootstrap-hub.sh
Hermes agent gateways	Per-profile processes (each writes to v2)	ACTIVE	hermes-gateway-{sergio,hr-agent,...}

→ One system is actually doing the work. Two are inert. One was thrashing.

The cross-contamination worry — and why it was moot

THE CONCERN

Why we thought agents were sharing memory

- Shared ~/.hermes/.env across every agent
- One MEMOS_USER_ID=ceo, one MEMOS_CUBE_ID=ceo-cube
- One API key — server sees one identity for every agent
- Server's cube-ownership check can't distinguish callers
- **Therefore: every write lands in the same cube**

THE REALITY

v1 was effectively unused all along

- memos-toolset (v1 client) was disabled on 2026-05-12
- Profiles already switched to provider: memtensor (v2)
- Probed v1 cube directly: text_mem=[], total_nodes=0
- v2 uses a different isolation model — namespace tuple
- **Net effect: zero actual contamination, ever**

ROW - LEVEL TENANCY · NO CUBES · NO API KEYS

How v2 isolation works

Part 2

Row-level multi-tenancy

Not cubes. A namespace tuple stamped on every row, checked at every read.

core/runtime/namespace.ts

```
// every read is filtered through this
function isVisibleTo(row, caller) {
  if (row.share_scope !== 'private') return true;
  return (
    row.owner_agent_kind === caller.agentKind &&
    row.owner_profile_id === caller.profileId
  );
}
```

```
// every row carries these columns
owner_agent_kind  TEXT NOT NULL
owner_profile_id  TEXT NOT NULL
share_scope       TEXT DEFAULT 'private'
```

T H E P R I M I T I V E

(agentKind, profileId, workspaceId)

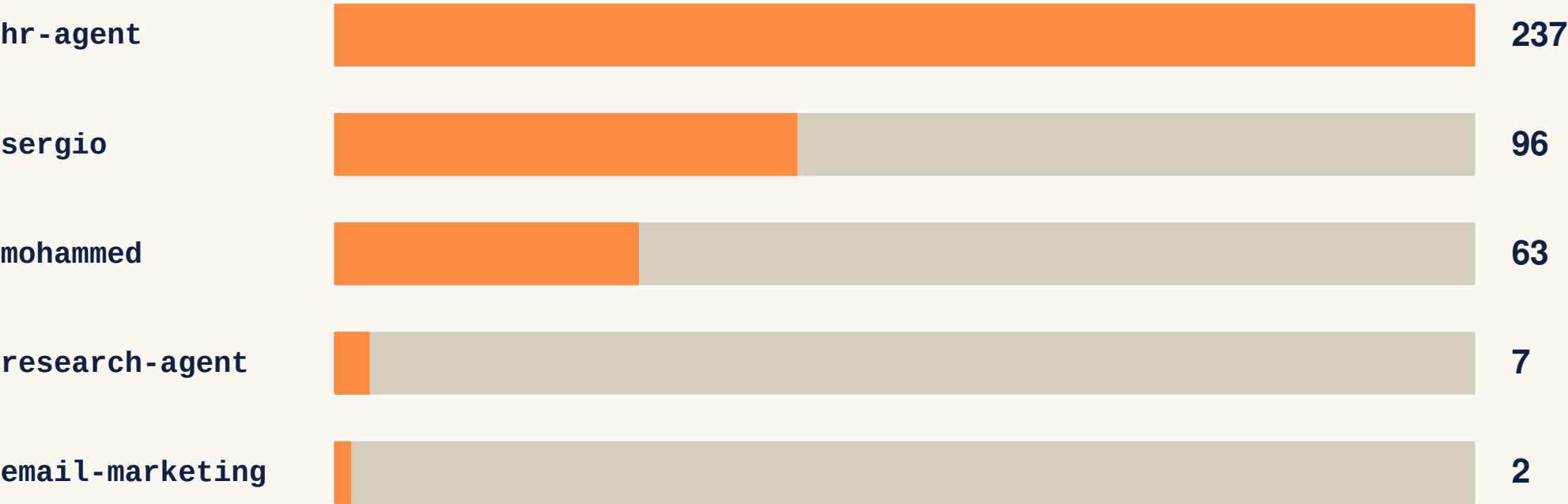
Plus a share_scope column on every row. Default is private. The plugin derives profileId from \$HERMES_HOME — so each agent process gets its own namespace automatically as long as it launches with its own profile dir.

W H Y I T W O R K S

- One SQLite, many tenants — no cubes, no per-agent files
- Enforced at every retrieval, not by trust
- No server, no API, no credential management

Isolation is working — verified

```
SELECT owner_profile_id, COUNT(*) FROM traces GROUP BY owner_profile_id
```



✓ No "default" bucket — every agent's writes land under its own profileid. The earlier worry was real for v1, but not for v2.

share_scope: intent, reality, and our policy

Four scope values exist. Today only one filters. We still tag rows with intent — when the hub ships, the data already knows where it should go.

Scope	Design intent	Today's behavior	As designed
private	Visible only to the writing profile	Only one that actually filters	✓
local	Visible across profiles on this machine	Visible to all (same as public)	✗
public	Visible to everyone, everywhere	Visible to all	✓
hub	Visible across machines via hub sync	Visible to all — hub never wired	✗

Our policy by table

Table	Scope	Effect
traces · episodes · policies	private	Raw experience stays per-agent — turns, sessions, candidate patterns
world_model · skills	local	Crystallised know-how shared across the team — attribution preserved via owner_profile_id

EXTRACTION · CRYSTALLISATION · EMBEDDING

Models

Part 3

Three models, three jobs

Different parts of the pipeline want different temperaments. We split the slots accordingly.

Stage	Model	Why this one
Extraction / summarisation (per-turn capture)	DeepSeek V3 deepseek-chat	Non-thinking model. Other reasoners pollute extraction with <think> tags that break the parser. Constrained generation — bigger isn't better. Cheap enough to never think about cost.
Skill crystallisation (L2 → active induction)	MiniMax M2.7 Anthropic-compatible endpoint	Synthesis-heavy. Reasoning helps. Output post-processed into YAML — <think> tags get stripped cleanly. Lower call frequency means cost matters less.
Embedding (trace summary + semantic search)	BGE-large-en-v1.5 Xenova/bge-large-en-v1.5 (local, 1024d)	Beat both MiniLM and gemini-embedding-2 on our actual data. Local — no API, no rate limits, no privacy footprint. See benchmark, next slides.

Embedder benchmark: how we tested

01

Pull real traces

200 trace summaries from the live v2 DB (read-only). Mixed profiles, real content — embedder choice, file transfer, idealista, plugin architecture.

02

Embed with all three

MiniLM-L6-v2 (current), BGE-large-en-v1.5 (proposed local), gemini-embedding-2 (proposed cloud). BGE used recommended query prefix.

03

Query set + retrieval

8 realistic memory queries. For each: top-5 retrieval from every model. Cosine on L2-normalized vectors.

04

LLM-as-judge

DeepSeek V3 rated each (query, snippet) pair on 0/1/2 relevance. Judged the UNION of top-5 across all models — no free passes.

METRICS: Precision@5 · MRR · nDCG@5 · pairwise top-5 overlap (Jaccard) · top-1 cosine similarity

RESULTS

BGE-large wins everything

+14% MRR over MiniLM, +4% MRR over Gemini — and free, local, no network.

Model	Dim	P@5	MRR	nDCG@5	Runtime	Cost	Verdict
MiniLM-L6-v2	384	0.450	0.688	0.628	Local (Xenova)	Free	baseline
BGE-large-en-v1.5	1024	0.500	0.781	0.725	Local (Xenova)	Free	WINNER
gemini-embedding-2	3072	0.425	0.750	0.662	Google API	Free tier / \$0.20-M	strong but costs

Δ vs MiniLM

+11% P@5 · +14% MRR · +15% nDCG@5

Δ vs Gemini-2

+18% P@5 · +4% MRR · +10% nDCG@5

Reading the numbers

P@5 — fraction of top-5 results genuinely relevant. MRR — reciprocal rank of the first relevant hit (higher = relevant items shown sooner). nDCG@5 — quality of ranking, discounted by position.

Why BGE-large is the pick

Q U A L I T Y

Top of every quality metric

P@5, MRR, nDCG@5 — all three. Idealista cluster and skill-crystallisation queries saw it find content the others missed entirely.

L O C A L

No API dependency

Runs in the Hermes process via Xenova/Transformers.js ONNX. No network round-trips, no rate limits, no Google account to manage.

P R I V A C Y

Memory never leaves the box

Every other option meant agent memory content was sent to a third-party API. BGE-large keeps it on the machine where it belongs.

F O O T P R I N T

Acceptable resource cost

~1.5 GB RAM working set; machine has ~9 GB free. ~150–300 ms per embedding on CPU. The hook is async — doesn't block the agent reply.

STAY ON V2 · FEWER MOVING PARTS

The verdict & cleanups

Part 4

Decision: stay on v2 for now

VERDICT

Single-tier: v2 plugin only

*Revisit v1 when multi-machine deployment becomes a real need,
or when an external agent runtime needs to hit the same memory.*

01

Single-developer research rig

We're optimizing for iteration speed, not multi-tenant production.

02

The UI matters

Memory browser + skill library make the system actually inspectable.

03

Auto-skill crystallization

Reward backprop → L2 promotion → YAML skill is genuinely novel and only in v2.

04

Isolation already works

Per-profile row tagging is enforced, verified, sufficient.

Cleanups: what we removed

Fewer moving parts. Each removal is reversible — none are destructive deletes, just service stops and config swaps.

memos-hub.service

DISABLED

Was thrashing in an auto-restart loop, calling bootstrap-hub.sh which tries to launch a hub mode the plugin doesn't implement. Zero functional impact.

memos-server.service (v1)

STOPPED

Healthy but unused — cube was empty, memos-toolset client already disabled. Frees Qdrant + Neo4j + Python memory. Restart anytime if we revisit v1.

gemini-embedding-2 config

REPLACED

Briefly swapped in during the embedder eval. Beaten by BGE-large on every quality metric. Reverted to local embedder before any data was written.

Auto-promoter cron

DISABLED

Orphan Sprint-3 cron rewriting share_scope=local every 15 min. Bypassed the plugin entirely via direct SQLite. Disabled in crontab, worktree retired.

A N O R P H A N C R O N F R O M S P R I N T 3

The auto-promoter root cause

Part 5

Every 15 min, share_scope went back to local

Manual demotion held for 14 minutes, then bulk-reverted. Symptom looked like the plugin, but wasn't.

1 Symptom

Manually flipped traces to private. Within 15 min, all rows back to local. Reproducible.

2 Forensic SQLite triggers

tools/forensic-audit.sql installs no-op AFTER-UPDATE triggers logging share_scope writes to a side table. Non-modifying — safe.

3 Next cycle, caught the actor

Bulk UPDATE pattern with no plugin sql_text — written directly via the SQLite CLI. Plugin's HTTP API + namespace filter bypassed entirely.

4 Traced to a cron

crontab -l revealed: `*/15 * * * * python3 .claude/worktrees/nice-mclaren-13f017/scripts/promote-memos-shares.py`

5 It was Paperclip/CEO scaffolding

Sprint 3 (2026-05-12) added it so the orchestrator could read across cubes. We retired Paperclip 5 days later — the cron lived on.

6 Disabled, not deleted

Commented out in crontab with rationale. Forensic triggers stay installed as a tripwire for re-emergence.

1 3 F I L E S · 5 L O G I C A L G R O U P S · N O U P S T R E A M R E P O

Plugin patches



Part 6

13 files patched, applied via postinstall hook

@memtensor/memos-local-plugin@2.0.0 ships with no public repo — patches live in tools/plugin-patches/ and re-apply on every npm install.

#	Logical change	Files	What it does
1	Shared-skill attribution	4	renderSkill appends '(learned by <profile>)' so agents see which profile crystallised a shared skill
2	Per-request namespace override	4	AsyncLocalStorage context parsed from X-As-Profile / ?as_profile=, applied at SQL time in trace pagination
3	Bundled-viewer UI overlay	2	Floating 'view as <profile>' picker injected into web/dist — intercepts fetch() to add the header
4	Skill packager default share-scope	1	New skills default share_scope=local (was private) — no more manual promotion after crystallisation
5	Named-speaker memory summaries	2	Summarizer reads MEMOS_HUMANS env, attributes speech to actual names — open-world (handles + self-intros)

Run: `bash tools/postinstall-patches.sh` · Verify: `bash tools/postinstall-patches.sh --check`

Patch sources live at tools/plugin-patches/ — a 1:1 mirror of every file we changed.

Per-agent viewer toggle

One daemon, one DB. Any agent's perspective on demand.

SERVER · ALS namespace

core/runtime/request-namespace.ts (new)

```
const als = new AsyncLocalStorage<RuntimeNamespace>();

export function runWithRequestNamespace(ns, fn) {
  return als.run(ns, fn);
}
export function getRequestNamespace() {
  return als.getStore();
}
```

http.ts wraps dispatch() in runWithRequestNamespace(ns, ...). memory-core.ts prefers ALS over the daemon namespace. traces.ts applies it at SQL time so paginated turn-keys match the requested profile.

CLIENT · fetch interceptor

web/dist/hermes-profile-switcher.js (new)

```
const orig = window.fetch.bind(window);
window.fetch = (input, init) => {
  if (!currentProfile) return orig(input, init);
  const headers = new Headers(init?.headers);
  headers.set('X-As-Profile', currentProfile);
  return orig(input, { ...init, headers });
};
```

Loaded via a <script> tag appended to web/dist/index.html. Lives outside the React bundle — survives bundle rebuilds and doesn't touch shadow-DOM internals.

Active at <https://tower.taila4a33f.ts.net/> — top-right dropdown.

Names from signal, not from a roster

Agents see messages from outside parties too — clients, leads, group chats, quoted email. The list is help, not a gate.

MEMOS_HUMANS (in ~/.hermes/.env)

```
MEMOS_HUMANS=Sergio:sergiopalacio96|sergiop,Krati,Mohammed:moh,Arinze
```

Format: Name[:handle1|handle2|...], ... · Name-only entries are fine — they still help when the body mentions the name.

How the summarizer infers the speaker

Signal	Example	Resulting summary
Gateway-prepended handle	[sergiopalacio96] can you check the cron?	Sergio asked to verify the cron schedule.
Self-introduction in body	Hi, I'm Mark from acme.io — want a quote on ...	Mark from acme.io requested a quote.
Quoted email From: header	From: anna@bizpartner.co	Anna requested a renewal.
No identifiable speaker	(direct message body with no marker)	Speaker reference omitted — never invented.

Files: core/capture/summarizer.ts · bridge.cts (dotenv loader so the daemon sees ~/.hermes/.env on its own)

I N S P E C T O R S · S T R E S S T E S T S · 3 D V I E W E R S

Tooling

Part 7

Inspect anything. Audit everything.

Tool	Purpose	Common invocation
tools/memos-explorer.py	Cross-agent CLI: list profiles, traces, skills, policies, world facts. Vector-space stats, similarity search, project per profile.	python3.12 tools/memos-explorer.py profiles python3.12 tools/memos-explorer.py search 'auth' --top 5
tools/forensic-audit.sql	Installs AFTER-UPDATE/INSERT/DELETE triggers logging every share_scope write to share_scope_audit. Non-modifying tripwire.	sqlite3 ~/.hermes/memos-plugin/data/memos.db < \ tools/forensic-audit.sql python3.12 tools/memos-explorer.py audit
tools/stress-test.sh	36 end-to-end checks: patch presence, daemon health, share-scope policy, per-agent visibility, embedding dims, capture pipeline.	bash tools/stress-test.sh # expect: 36 passed, 0 failed
tools/postinstall-patches.sh	Idempotent applier of all 13 plugin patches + symlinks for UMAP / graph viewers. Run after every npm install of the plugin.	bash tools/postinstall-patches.sh bash tools/postinstall-patches.sh --check

2D Plotly + 3D deck.gl

Sanity-check what the BGE-large embedder actually believes — interactively.

2 D · Plotly

`tools/umap-viewer.html`

Static UMAP projection. Color by profile, hover shows summary, click opens full trace in the side panel.

`https://tower.taila4a33f.ts.net/umap-viewer.html`

Reads `tools/vec-map.json`

Best for: a quick read on cluster topology, identifying outliers.

3 D · deck.gl · GPU

`tools/umap-3d-viewer.html`

Orbit camera, GPU-rendered point cloud, drag-to-rotate. Cross-agent rows wear white halos so they pop against any profile color.

`https://tower.taila4a33f.ts.net/umap-3d-viewer.html`

Reads `tools/vec-map-3d.json`

Best for: spotting bridges between profiles, seeing structure the 2D projection flattened.

UX touches

Profile palette redesigned so no agent color collides with the ember accent. · Cross-agent indicator switched from orange tint to white halo. · Persistent plain-language 'How to read this map' panel.

WHAT'S GREEN · WHAT'S NEXT · WHAT TO READ

Final state & forward

Part 8

Where we landed: config.yaml

```
~/.hermes/memos-plugin/config.yaml
embedding:
  provider:    local
  model:       Xenova/bge-large-en-v1.5      # +11-18% over MiniLM / Gemini-2
  dimensions: 1024

llm:
  provider:    openai_compatible
  endpoint:    https://api.deepseek.com
  model:       deepseek-chat                 # extraction / summarisation

skillEvolver:
  provider:    anthropic
  endpoint:    https://api.minimax.io/anthropic
  model:       MiniMax-M2.7                  # crystallisation / L2 induction

hub:
  enabled:     false                        # feature unimplemented unstream
```

Isolation is per-profile via \$HERMES_HOME → profileId. No cubes, no shared credentials. Verified against the running DB (36 / 36 stress-test checks green).

All green. Durable.

36 / 36

stress-test checks

13 / 13

plugin patches verified

1 / 1

auto-promoter killed

0

manual steps after npm install

Keep it green

After every npm install/update of @memtensor/memos-local-plugin	bash tools/postinstall-patches.sh
After any change to core daemon code / share policy	bash tools/stress-test.sh
If share_scope ever drifts unexpectedly again	python3.12 tools/memos-explorer.py audit

Not blocking. Worth doing.

Three lines we picked up but didn't pull all the way through.

Cluster summarisation in the UMAP viewers

Ask DeepSeek V3 to one-line each HDBSCAN cluster (≈ 31 calls). Render labels in 2D + 3D viewers so the topology becomes self-explanatory.

`tools/umap-viewer.html` · `tools/umap-3d-viewer.html`

Per-profile MEMOS_HUMANS

Today MEMOS_HUMANS is a single env var. Expand to MEMOS_HUMANS__<profile> so each agent can carry its own alias map.

`core/capture/summarizer.ts`

Backfill historical traces with the named-speaker prompt

Existing summaries still say 'the user'. A re-summarisation pass against the new prompt would put names into older embeddings and align them with new ones.

`tools/memos-explorer.py` (new `resummarize` subcommand)

What to read · what to run

Canonical artefacts

`CLAUDE.md` (repo root)

Active sprint + memory rules every new agent reads first

`memos-setup/learnings/ 2026-05-17-v2-only-bge-shares.md`

Canonical decision doc — backend, embedder, share policy, rollback

`tools/plugin-patches/`

1:1 mirror of every patched file

`tools/forensic-audit.sql`

Non-modifying SQLite triggers — install once, audit forever

`https://tower.taila4a33f.ts.net/`

Memory viewer (password-gated). Top-right drop-down switches lens to any agent.

Five things to try

1

Open the 3D viewer

`https://tower.taila4a33f.ts.net/umap-3d-viewer.html`

2

Send a message with a handle

`[sergiopalacio96] what's the embedder cost again?`

3

Audit `share_scope` writes

`python3.12 tools/memos-explorer.py audit`

4

Run the full stress test

`bash tools/stress-test.sh`

5

Switch viewer perspective

`visit the viewer · pick a profile from the top-right drop-down`

E N D O F B R I E F

One day. One memory system.

*v2 plugin chosen, BGE-large wired, auto-promoter killed,
per-agent viewer shipped, speaker attribution open-world.*

Questions worth asking next session:

- How do skills propagate when a new profile spins up — instant via shared layer, or does it need a recall pass?
- Should cluster labels in the UMAP viewers be cached, or recomputed every embedding-set rebuild?
- What's the right TTL on a private trace before it's eligible for archival to a cold layer?